

Content production and delivery for interactive multimedia services — a new approach

R Walker, P Foster and S Banthorpe

An increasing number of multimedia services are being launched, many of which are Internet based. New, reliable, low-cost services must generate customer demand and sustain customer satisfaction.

One of the technical directors from the computer games company, Sega, has been quoted as saying that ‘the games platform is only an enabler; apart from price, three further things are required: content, content, content’. This statement holds good for any interactive multimedia service, the content is what the consumer is seeking to access — and indeed most of what the consumer is paying for.

This paper presents a framework covering the authoring and distributed handling of content and its associated metadata (i.e. data associated with the content which is used by the application and its platform but not necessarily viewed by the end user, for example, a movie rating). It focuses primarily on the specification of both content and metadata and discusses the relationships between processes collectively operating as a generic tool-set.

1. Introduction

Projects such as BT Interactive TV (BTiTV), BT Touchpoint and Campus World [1, 2] have generated considerable experience in developing and delivering multimedia services. In doing so a number of challenges have been identified, many of which are content related.

A research programme was set up to address some of these problems and generate innovative solutions. One of its work packages is focused on tackling content-specific problems and its main aims are:

- to identify a set of generic processes that can be used during the production and delivery of multimedia content to any application running on a server,
- to specify the interfaces between these processes and to develop a concept demonstrator that can be used to test these interfaces and develop them further.

Until now, it has typically been necessary to develop custom processes and tools to manage content for each new service. This can be costly and time consuming. Working on a set of generic processes should remove the need for ‘stove-pipe’ solutions and enable reuse of components between services.

A wide variety of proprietary tools exist which may be used during content production and delivery (for example MPEG encoding tools). However, such tools are not always designed to work in an integrated way and often only provide solutions for specific tasks. By defining a set of process interfaces, it is possible to define a tool-set that provides a complete solution. It is also possible to remove the need to define a specific implementation. This leaves the tool-set open, providing the opportunity for different implementations.

Some of the key processes investigated to successfully manage multimedia content are:

- specification of content and its associated metadata, using a proposed content and metadata specification language (CMSL) within the application authoring environment to produce a standardised metadata declaration format — this is important if commercially available authoring systems are to incorporate tools to create it,
- tracking and scheduling of content, i.e. lifetime management of content — tracking is concerned with managing content as it progresses through the production process and finally becomes authorised and

Content production and delivery for interactive multimedia services — a new approach

R Walker, P Foster and S Banthorpe

An increasing number of multimedia services are being launched, many of which are Internet based. New, reliable, low-cost services must generate customer demand and sustain customer satisfaction.

One of the technical directors from the computer games company, Sega, has been quoted as saying that ‘the games platform is only an enabler; apart from price, three further things are required: content, content, content’. This statement holds good for any interactive multimedia service, the content is what the consumer is seeking to access — and indeed most of what the consumer is paying for.

This paper presents a framework covering the authoring and distributed handling of content and its associated metadata (i.e. data associated with the content which is used by the application and its platform but not necessarily viewed by the end user, for example, a movie rating). It focuses primarily on the specification of both content and metadata and discusses the relationships between processes collectively operating as a generic tool-set.

1. Introduction

Projects such as BT Interactive TV (BTiTV), BT Touchpoint and Campus World [1, 2] have generated considerable experience in developing and delivering multimedia services. In doing so a number of challenges have been identified, many of which are content related.

A research programme was set up to address some of these problems and generate innovative solutions. One of its work packages is focused on tackling content-specific problems and its main aims are:

- to identify a set of generic processes that can be used during the production and delivery of multimedia content to any application running on a server,
- to specify the interfaces between these processes and to develop a concept demonstrator that can be used to test these interfaces and develop them further.

Until now, it has typically been necessary to develop custom processes and tools to manage content for each new service. This can be costly and time consuming. Working on a set of generic processes should remove the need for ‘stove-pipe’ solutions and enable reuse of components between services.

A wide variety of proprietary tools exist which may be used during content production and delivery (for example MPEG encoding tools). However, such tools are not always designed to work in an integrated way and often only provide solutions for specific tasks. By defining a set of process interfaces, it is possible to define a tool-set that provides a complete solution. It is also possible to remove the need to define a specific implementation. This leaves the tool-set open, providing the opportunity for different implementations.

Some of the key processes investigated to successfully manage multimedia content are:

- specification of content and its associated metadata, using a proposed content and metadata specification language (CMSL) within the application authoring environment to produce a standardised metadata declaration format — this is important if commercially available authoring systems are to incorporate tools to create it,
- tracking and scheduling of content, i.e. lifetime management of content — tracking is concerned with managing content as it progresses through the production process and finally becomes authorised and

available for use by the service, while scheduling is concerned with the active part of the content life cycle, from when content is ready to be transferred to the live server, through to expiry,

- testing of content and metadata is one of the most difficult processes to automate because of its subjective nature — testing must be performed as early in the life cycle as possible so that errors can be corrected without unduly affecting the schedule.

This paper provides an overview of the generic processes and discusses how these processes may interact in a multi-user, heterogeneous environment. In addition it will focus on the specification of content and its associated metadata. A more detailed description of the individual processes can be found in Searby [3].

2. Overview of top level processes

The first step in designing the content process is to recognise the distinction between the application and its content. With the use of languages such as Sun's Java [4] this distinction is not necessarily a simple separation of executable code from data files. The distinction is really between the relatively static part which is the application framework and the more dynamic part which is the information to be delivered — the content.

The production and delivery of content is considered in terms of its life cycle phases as shown in Fig 1.

The first four phases are associated primarily with the 'production' part of the life cycle. **Acquisition** is the procurement of any piece of physical media or metadata associated with it. The media would ordinarily be raw content, e.g. a photograph or a video tape. Once the media has been acquired it may require **transcoding** into a format suitable for the target application, e.g. scanning of a photograph or MPEG compression of video. In cases where

the transcoded material is already in the correct format, this stage can be omitted. The transcoded material must then go through a **quality assurance** phase to ensure it meets the expected standard, e.g. checking that the artefacts on compressed video are sufficiently low. When the media has passed the quality assurance phase and its associated metadata has been acquired, the item can be submitted for **validation, verification and authorisation**. This will ensure that the media is suitable for the application for which it is intended and that all metadata for the offering (i.e. a set of purchasable items) is complete and correct. Once tests are completed successfully, the item is marked as authorised for use by the target application.

The last four phases are primarily concerned with the 'active' part of the life cycle. **Delivery** of the content and metadata to the target application must take place by a specified date. The content is delivered to the application 'in-box' for **loading** into the application. Once loading is complete, the content is available to end-users. End-users are prevented from accessing the content at the specified **expiry** time, i.e. after it has served its 'useful' purpose. Finally, content is deleted from the application at the **removal** phase.

Throughout its life cycle, content is defined using one of two packages, content items and content item elements [5].

A content item element (CIE) is the smallest (and indivisible) content component and can be part of one or more content items. Examples of such elements are still pictures and video clips. A content item (CI) is a collection of CIEs (and/or other CIs) that will collectively form a more complex, and often more abstract, piece of content, for example, a movie 'offering' which may comprise the actual video, a trailer and a detail screen CI. The detail screen CI could comprise a synopsis, a still picture and a background (see Fig 2).

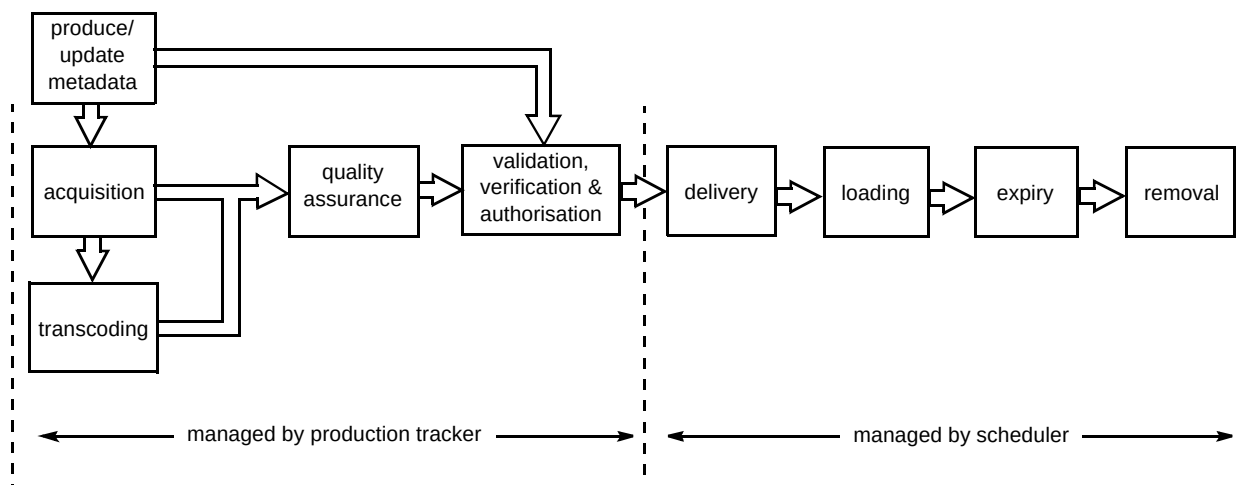


Fig 1 Content life cycle.

available for use by the service, while scheduling is concerned with the active part of the content life cycle, from when content is ready to be transferred to the live server, through to expiry,

- testing of content and metadata is one of the most difficult processes to automate because of its subjective nature — testing must be performed as early in the life cycle as possible so that errors can be corrected without unduly affecting the schedule.

This paper provides an overview of the generic processes and discusses how these processes may interact in a multi-user, heterogeneous environment. In addition it will focus on the specification of content and its associated metadata. A more detailed description of the individual processes can be found in Searby [3].

2. Overview of top level processes

The first step in designing the content process is to recognise the distinction between the application and its content. With the use of languages such as Sun's Java [4] this distinction is not necessarily a simple separation of executable code from data files. The distinction is really between the relatively static part which is the application framework and the more dynamic part which is the information to be delivered — the content.

The production and delivery of content is considered in terms of its life cycle phases as shown in Fig 1.

The first four phases are associated primarily with the 'production' part of the life cycle. **Acquisition** is the procurement of any piece of physical media or metadata associated with it. The media would ordinarily be raw content, e.g. a photograph or a video tape. Once the media has been acquired it may require **transcoding** into a format suitable for the target application, e.g. scanning of a photograph or MPEG compression of video. In cases where

the transcoded material is already in the correct format, this stage can be omitted. The transcoded material must then go through a **quality assurance** phase to ensure it meets the expected standard, e.g. checking that the artefacts on compressed video are sufficiently low. When the media has passed the quality assurance phase and its associated metadata has been acquired, the item can be submitted for **validation, verification and authorisation**. This will ensure that the media is suitable for the application for which it is intended and that all metadata for the offering (i.e. a set of purchasable items) is complete and correct. Once tests are completed successfully, the item is marked as authorised for use by the target application.

The last four phases are primarily concerned with the 'active' part of the life cycle. **Delivery** of the content and metadata to the target application must take place by a specified date. The content is delivered to the application 'in-box' for **loading** into the application. Once loading is complete, the content is available to end-users. End-users are prevented from accessing the content at the specified **expiry** time, i.e. after it has served its 'useful' purpose. Finally, content is deleted from the application at the **removal** phase.

Throughout its life cycle, content is defined using one of two packages, content items and content item elements [5].

A content item element (CIE) is the smallest (and indivisible) content component and can be part of one or more content items. Examples of such elements are still pictures and video clips. A content item (CI) is a collection of CIEs (and/or other CIs) that will collectively form a more complex, and often more abstract, piece of content, for example, a movie 'offering' which may comprise the actual video, a trailer and a detail screen CI. The detail screen CI could comprise a synopsis, a still picture and a background (see Fig 2).

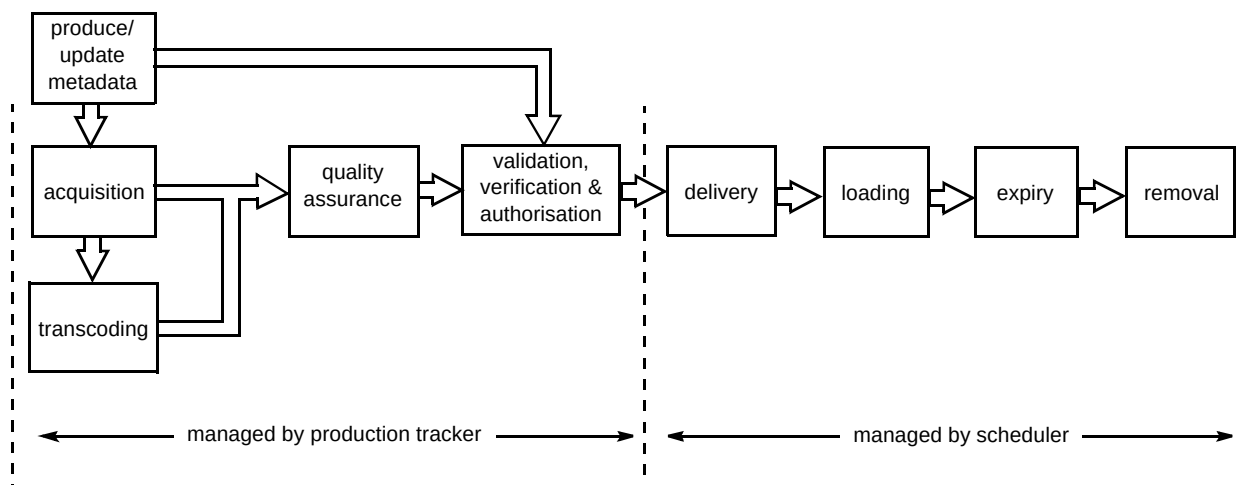


Fig 1 Content life cycle.

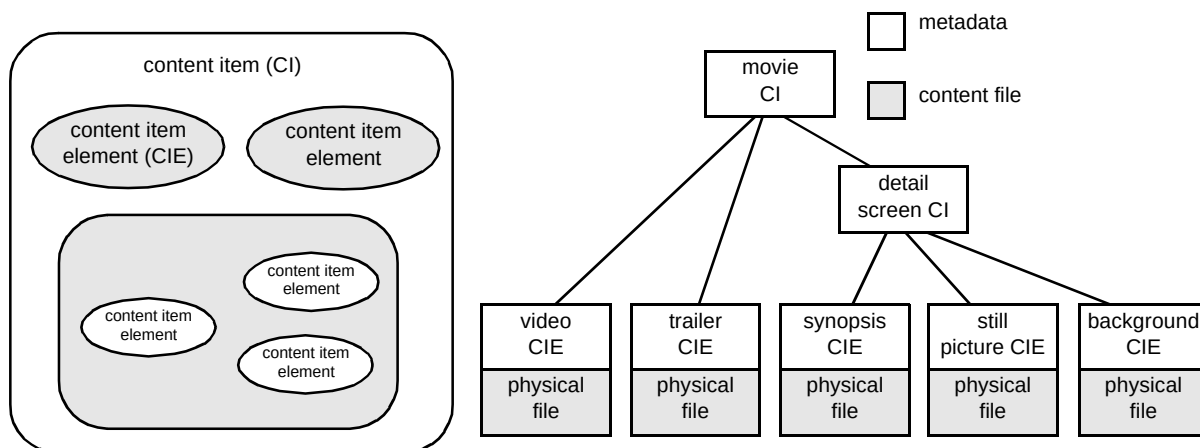


Fig 2 Illustration of a content item.

Figure 3 shows the interaction between the processes needed in the automation of content production and delivery through the life cycle stages. The processes are divided into three domains — content production, application management and service platform.

In the content production domain the processes involved are collectively known as a ‘content production tool set’ and manage content through the production part of the life cycle. A tool set graphical user interface (GUI) allows the users to invoke the other domain processes. Physical media,

both raw and transcoded, are stored in a media store. Third-party tools may be used to transcode raw media into the desired format (e.g. MPEG encoding).

Content metadata is collated via a content metadata editor (CME). A CME allows new content metadata to be created against a predefined specification which is defined using the proposed content and metadata specification language (CMSL). This language is discussed in detail, later in the paper. Metadata persistence is provided via the library process.

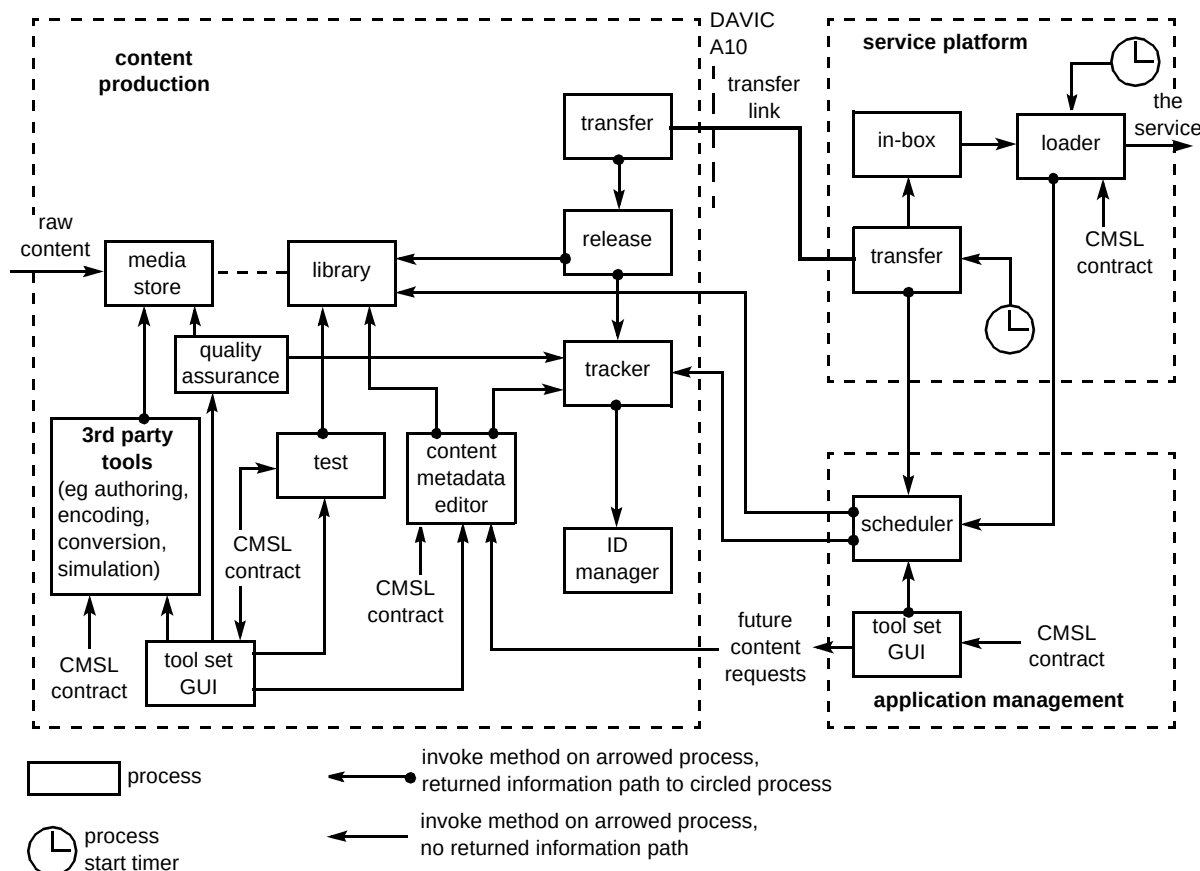


Fig 3 Content production and delivery — process overview.

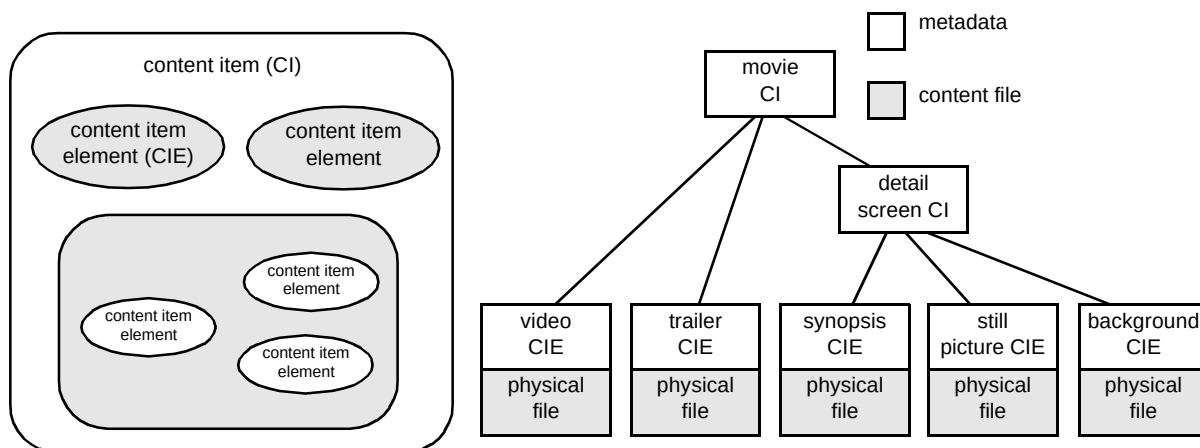


Fig 2 Illustration of a content item.

Figure 3 shows the interaction between the processes needed in the automation of content production and delivery through the life cycle stages. The processes are divided into three domains — content production, application management and service platform.

In the content production domain the processes involved are collectively known as a ‘content production tool set’ and manage content through the production part of the life cycle. A tool set graphical user interface (GUI) allows the users to invoke the other domain processes. Physical media,

both raw and transcoded, are stored in a media store. Third-party tools may be used to transcode raw media into the desired format (e.g. MPEG encoding).

Content metadata is collated via a content metadata editor (CME). A CME allows new content metadata to be created against a predefined specification which is defined using the proposed content and metadata specification language (CMSL). This language is discussed in detail, later in the paper. Metadata persistence is provided via the library process.

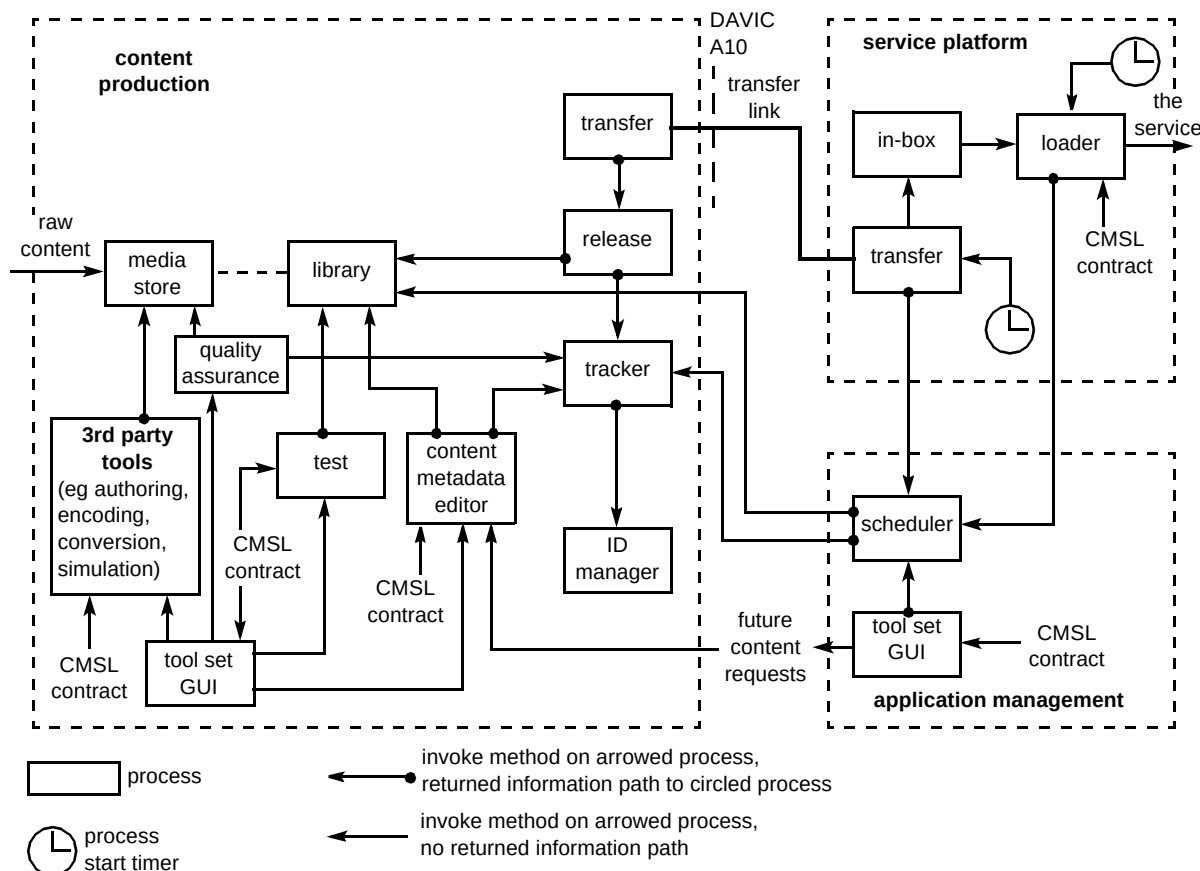


Fig 3 Content production and delivery — process overview.

When a new piece of content is created, the CME must also request the allocation of a unique ID via the tracker. The tracker creates a new entry in its database which enables it to 'track' the content's progression through the production part of the life cycle. Content is automatically verified by the test process and authorised for release.

The application management domain and the service platform are used to manage content through the active part of its life cycle. The scheduler process, in the application management domain, selects content authorised for release and instructs the transfer process on the service platform to retrieve them into its 'in-box'.

Content is retrieved by interaction between the scheduler, transfer and release processes to create specific containers of packaged CIs and CIEs. The actual delivery of containers may be performed electronically over a DAVIC A10 interface [5] or via some other form of physical transfer medium.

On reaching the service platform, the container is unpacked by a content loader which distributes the media and metadata at a scheduled time. As the loader is application specific, it understands the internal structure of the application and can intelligently distribute the content to the relevant locations on the service platform. The loader decouples the specific requirements of an application from the generic tool-set.

2.1 The need for distribution

It is essential, when attempting to automate the multimedia content production and delivery environment with a set of generic processes, that they reside within a suitable open and distributed object infrastructure for the following reasons:

- ownership of individual processes could be spread throughout an organisation or even across organisations, e.g. one organisation may be responsible for MPEG-encoding raw video, another may be responsible for creating content items,
- there may be a need for some processes to have multiple instances to support multiple users across a network (e.g. CME) which share a common instance of another process (e.g. tracker),
- individual processes have different platform requirements, e.g. the library may need to run on a platform that is capable of hosting a particular type of database, and each process may also require different processing power,

- distribution provides flexibility within the tool-set to cope with different process ownership scenarios, e.g. quality assurance may be performed by a remote, independent third party.

2.2 A proposed solution

Since 1989 the Object Management Group (OMG) has been committed to defining an open middleware solution (or 'object bus' as it is sometimes referred to) for distributed objects called the Common Object Request Broker Architecture (CORBA) [6]. There are a number of CORBA-compliant object broker implementations available, e.g. SOM (system object model) from IBM, ORBPlus from HP, ObjectBroker from Digital, NEO from Sun and ORBIX from IONA.

Using CORBA to distribute the processes identified in Fig 3 offers a number of important advantages. It is a well-established standard with several important CORBA utility services already available, e.g. the CORBA naming service can be used to locate other processes in the network. CORBA interfaces are defined using an interface definition language (IDL) which is designed to be program language independent. This enables CORBA to support distributed computing in a heterogeneous environment. IDL interpreters are available to convert IDL into an equivalent programming language for implementation, e.g. C++ or Java.

2.3 Client/server relationships

A server process is essentially a remote object with defined procedures that a client process can call remotely, just as if it resided locally. Some server processes can act as client processes too when they access other remote server processes. The term 'servient' is sometimes used to refer to these types of server process. Within the tool set, the library, tracker, ID manager, release and scheduler have been identified as server processes, while test, quality assurance, content metadata editor, transfer and loader are identified as client processes.

Before a client process can invoke a function on a server process it must obtain a remote reference to it. This is normally obtained from a session-manager process. The session manager controls access to instance(s) of the server processes it manages.

2.4 Process interfaces

IDL interfaces define the public methods available between the client and server processes. As stated earlier these interfaces are portable between languages, operating

When a new piece of content is created, the CME must also request the allocation of a unique ID via the tracker. The tracker creates a new entry in its database which enables it to 'track' the content's progression through the production part of the life cycle. Content is automatically verified by the test process and authorised for release.

The application management domain and the service platform are used to manage content through the active part of its life cycle. The scheduler process, in the application management domain, selects content authorised for release and instructs the transfer process on the service platform to retrieve them into its 'in-box'.

Content is retrieved by interaction between the scheduler, transfer and release processes to create specific containers of packaged CIs and CIEs. The actual delivery of containers may be performed electronically over a DAVIC A10 interface [5] or via some other form of physical transfer medium.

On reaching the service platform, the container is unpacked by a content loader which distributes the media and metadata at a scheduled time. As the loader is application specific, it understands the internal structure of the application and can intelligently distribute the content to the relevant locations on the service platform. The loader decouples the specific requirements of an application from the generic tool-set.

2.1 The need for distribution

It is essential, when attempting to automate the multimedia content production and delivery environment with a set of generic processes, that they reside within a suitable open and distributed object infrastructure for the following reasons:

- ownership of individual processes could be spread throughout an organisation or even across organisations, e.g. one organisation may be responsible for MPEG-encoding raw video, another may be responsible for creating content items,
- there may be a need for some processes to have multiple instances to support multiple users across a network (e.g. CME) which share a common instance of another process (e.g. tracker),
- individual processes have different platform requirements, e.g. the library may need to run on a platform that is capable of hosting a particular type of database, and each process may also require different processing power,

- distribution provides flexibility within the tool-set to cope with different process ownership scenarios, e.g. quality assurance may be performed by a remote, independent third party.

2.2 A proposed solution

Since 1989 the Object Management Group (OMG) has been committed to defining an open middleware solution (or 'object bus' as it is sometimes referred to) for distributed objects called the Common Object Request Broker Architecture (CORBA) [6]. There are a number of CORBA-compliant object broker implementations available, e.g. SOM (system object model) from IBM, ORBPlus from HP, ObjectBroker from Digital, NEO from Sun and ORBIX from IONA.

Using CORBA to distribute the processes identified in Fig 3 offers a number of important advantages. It is a well-established standard with several important CORBA utility services already available, e.g. the CORBA naming service can be used to locate other processes in the network. CORBA interfaces are defined using an interface definition language (IDL) which is designed to be program language independent. This enables CORBA to support distributed computing in a heterogeneous environment. IDL interpreters are available to convert IDL into an equivalent programming language for implementation, e.g. C++ or Java.

2.3 Client/server relationships

A server process is essentially a remote object with defined procedures that a client process can call remotely, just as if it resided locally. Some server processes can act as client processes too when they access other remote server processes. The term 'servient' is sometimes used to refer to these types of server process. Within the tool set, the library, tracker, ID manager, release and scheduler have been identified as server processes, while test, quality assurance, content metadata editor, transfer and loader are identified as client processes.

Before a client process can invoke a function on a server process it must obtain a remote reference to it. This is normally obtained from a session-manager process. The session manager controls access to instance(s) of the server processes it manages.

2.4 Process interfaces

IDL interfaces define the public methods available between the client and server processes. As stated earlier these interfaces are portable between languages, operating

systems and networks. The interfaces shown in Fig 2 have been designed to be simple to avoid complex implementation requirements. For example, the library process interface is designed to allow a client process to:

- open and close a connection to the library database,
- store a newly created CI or CIE metadata object in the library database,
- update the contents of an existing CI or CIE,
- delete an existing CI or CIE,
- return a set of CIs or CIEs that match a given ID and/or set of specific field values.

All access rights to the Library process are controlled via a library session manager. A client process cannot obtain a reference to a library process unless authorised by the session manager.

3. Content and metadata specification language (CMSL)

3.1 Overview

CMSL is essentially used as a means of defining content specifications in a standardised format that can easily be interpreted by a calling function (e.g. the CME). It is important to clarify that a content specification is simply a template that models a piece of content, both in terms of its physical characteristics and its metadata requirements. For example, there may exist a specification for a movie offering and an instance of it would be the *Star Wars* movie. Only when specific values are assigned, is an instance of a content specification created. Although, in this respect, it is similar to a class in object-oriented technology, the specifications have additional parameters to define default values, constraints, etc. The use of standardised content specifications offers a number of key benefits:

- a user can create new content to different specifications via a common GUI that automatically prompts the user for the required data, thus simplifying the process,

- testing of content for correctness and completeness before being transferred to the service platform is automated,
- provision of a common interface for a service specific loader, thereby decoupling a generic process from a specific use of its output,
- portability across a range of authoring systems.

A CI specification comprises a generic set of data, a set of assignable attributes and a set of associations (i.e. component CI or CIEs).

A CIE specification, similarly, comprises a generic set of data, a set of assignable attributes and a media type (see Fig 4).

Generic data requirements include ID, title, description, etc. This data is typically required by all instances of a content specification.

Attributes are used to describe characteristics of a specific CI/CIE, for example, a 'movie' offering may use 'actor', 'rating' and 'genre' attributes. Alternatively, a home shopping application may include a 'shoes' offering which may use 'style', 'colour' and 'size' attributes.

Associations allow the creation of complex CMSL structures by assigning a set of CIs or CIEs to a parent CI. For example, a '*Star Wars* movie package' could comprise the following CIs — *Star Wars*, *The Empire Strikes Back* and *The Return of the Jedi* along with a number of CIEs (e.g. trailer, MPEG still background, etc). A CIE does not have associations as it is the smallest content item and as such the most basic CMSL structure.

A CIE must be of an assignable type. Its type defines the information required to build the desired physical piece of media. For example a CIE that is of type 'MPEG video' would need to be MPEG encoded. Additional characteristics of the 'MPEG video' type may include its bit rate, frame size, etc.

Relationships identify and define the links between a CI/CIE and its components. For example, a CI may have an 'actors' attribute assigned to it which could take several

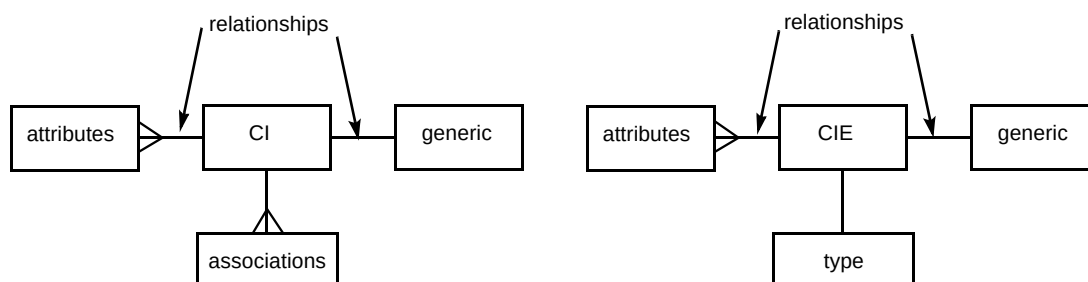


Fig 4 CI and CIE specification components.

systems and networks. The interfaces shown in Fig 2 have been designed to be simple to avoid complex implementation requirements. For example, the library process interface is designed to allow a client process to:

- open and close a connection to the library database,
- store a newly created CI or CIE metadata object in the library database,
- update the contents of an existing CI or CIE,
- delete an existing CI or CIE,
- return a set of CIs or CIEs that match a given ID and/or set of specific field values.

All access rights to the Library process are controlled via a library session manager. A client process cannot obtain a reference to a library process unless authorised by the session manager.

3. Content and metadata specification language (CMSL)

3.1 Overview

CMSL is essentially used as a means of defining content specifications in a standardised format that can easily be interpreted by a calling function (e.g. the CME). It is important to clarify that a content specification is simply a template that models a piece of content, both in terms of its physical characteristics and its metadata requirements. For example, there may exist a specification for a movie offering and an instance of it would be the *Star Wars* movie. Only when specific values are assigned, is an instance of a content specification created. Although, in this respect, it is similar to a class in object-oriented technology, the specifications have additional parameters to define default values, constraints, etc. The use of standardised content specifications offers a number of key benefits:

- a user can create new content to different specifications via a common GUI that automatically prompts the user for the required data, thus simplifying the process,

- testing of content for correctness and completeness before being transferred to the service platform is automated,
- provision of a common interface for a service specific loader, thereby decoupling a generic process from a specific use of its output,
- portability across a range of authoring systems.

A CI specification comprises a generic set of data, a set of assignable attributes and a set of associations (i.e. component CI or CIEs).

A CIE specification, similarly, comprises a generic set of data, a set of assignable attributes and a media type (see Fig 4).

Generic data requirements include ID, title, description, etc. This data is typically required by all instances of a content specification.

Attributes are used to describe characteristics of a specific CI/CIE, for example, a 'movie' offering may use 'actor', 'rating' and 'genre' attributes. Alternatively, a home shopping application may include a 'shoes' offering which may use 'style', 'colour' and 'size' attributes.

Associations allow the creation of complex CMSL structures by assigning a set of CIs or CIEs to a parent CI. For example, a '*Star Wars* movie package' could comprise the following CIs — *Star Wars*, *The Empire Strikes Back* and *The Return of the Jedi* along with a number of CIEs (e.g. trailer, MPEG still background, etc). A CIE does not have associations as it is the smallest content item and as such the most basic CMSL structure.

A CIE must be of an assignable type. Its type defines the information required to build the desired physical piece of media. For example a CIE that is of type 'MPEG video' would need to be MPEG encoded. Additional characteristics of the 'MPEG video' type may include its bit rate, frame size, etc.

Relationships identify and define the links between a CI/CIE and its components. For example, a CI may have an 'actors' attribute assigned to it which could take several

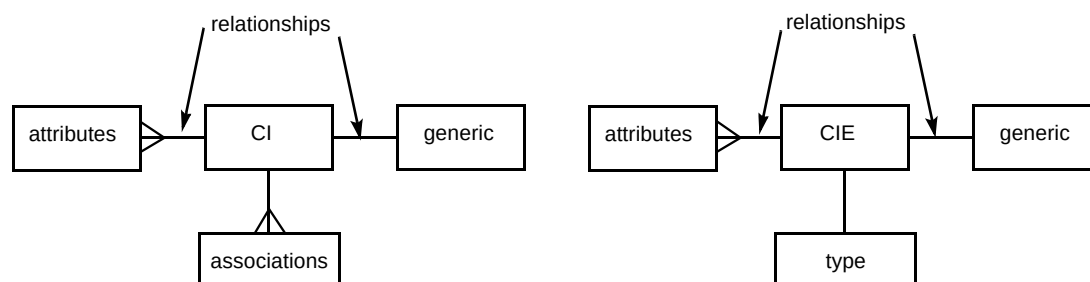


Fig 4 CI and CIE specification components.

values, i.e. there is a one-to-many relationship between the CI and its ‘actors’ attribute. Alternatively a movie package may contain several ‘movie’ offerings for which there would be a one-to-many relationship. CMSL allows these relationships to be defined in detail, including whether the relationship is mandatory or optional.

3.2 CMSL — a specification language

CMSL is based around a simple non-procedural language offering complete reusability, flexibility, portability and human readability for defining all types of multimedia content.

CMSL source files are parsed by an interpreter. The interpreter provides an application programming interface (API) which the other tool-set processes, including a service-specific loader, can call to obtain details of CMSL constructs.

The language consists of a number of major syntactic constructs, each of which is enclosed with its own unique keyword and common end marker, for example:

```
CIM_SPEC
...
END
```

Within the bounds of a keyword/end marker, ‘attributes’ can be declared to specify the details of the major construct. Parameters can also be associated with these attributes to define maximum/minimum values, default values, optional values, number of duplicate attribute occurrences, etc.

Six major CMSL syntactic constructs have been identified. Some of these are described in the example illustrated in Fig 5. It should be noted that CMSL reserved words are shown in **BOLD** characters.

Figure 5 illustrates how a typical CMSL structure and the representative CMSL could be designed. Parts (a), (b) and (c) illustrate how the final structure is built up. The structure in this example, while not necessarily complete, could be used to define movie type offerings. Each structure equates to a specific set of CMSL constructs. For instance, structure (a) defines the actual CIM_SPEC ‘movie’ including the associated generic data component. The order of CMSL constructs in Fig 5 is that required by the CMSL interpreter.

Structure (b) illustrates the inclusion of a number of attribute components — namely ‘actors’, ‘rating’, ‘genre’ and ‘price’ attributes. The dashed line between the movie CI and genre attribute components indicates that the relationship is optional; this is reflected in the CMSL by the

keyword ‘**OPTIONAL**’ in the **ATTRIBUTES_USED** construct. The use of an ‘optional’ keyword aids the automated testing process to determine if an item is complete, i.e. it can check for completeness of metadata for a given piece of content by comparing entries in the library against the CMSL specification to which the item was built. The ‘ratings’ attribute also illustrates the capability of CMSL to specify default values and a list of valid values, both of which improve efficiency and accuracy during metadata acquisition. In addition, they provide another level of confidence with respect to the integrity of the content’s metadata. The relationship between the movie CI and the ‘Actors’ attribute component is a one-to-many relationship, as indicated by the branched link. This indicates that a piece of content built to this specification may have many Actors assigned to it. For example, the *Star Wars* offering may have ‘Harrison Ford’, ‘Carrie Fisher’ and ‘Alec Guinness’ assigned to it. Again this is reflected in the CMSL by the keyword ‘**OCCURS**’. The numbers following ‘**OCCURS**’ further defines the relationship, i.e. in the example given it indicates that there must be at least one value (an actor) assigned, and a maximum of five values.

Structure (c) indicates the inclusion of a number of associations. The MPEG ‘Trailer’ association is again indicated as optional.

3.3 Generation of CMSL

The example structure illustrated in Fig 5 is a simple one. Far more complex structures are possible. The flexibility in defining these structures is vast and is dependent upon the designer’s expertise. As the structures become more complex, there arises a real need to manage them in a user friendly environment. In addition, the production of the appropriate CMSL could be both a potential bottle-neck to the system and error prone.

By providing an object-oriented user-interface tool that allows the user to build their required CMSL structures in a drag-and-drop environment, it may be possible to overcome these problems. The tool could be used to automatically generate the CMSL. This approach offers a number of key advantages:

- provision of a user-friendly environment for designing and updating CMSL structures,
- management of CMSL structure definitions promoting reuse across specifications,
- reduction in the required level of expertise, by abstracting them from the internal complexities of CMSL as a language,

values, i.e. there is a one-to-many relationship between the CI and its ‘actors’ attribute. Alternatively a movie package may contain several ‘movie’ offerings for which there would be a one-to-many relationship. CMSL allows these relationships to be defined in detail, including whether the relationship is mandatory or optional.

3.2 CMSL — a specification language

CMSL is based around a simple non-procedural language offering complete reusability, flexibility, portability and human readability for defining all types of multimedia content.

CMSL source files are parsed by an interpreter. The interpreter provides an application programming interface (API) which the other tool-set processes, including a service-specific loader, can call to obtain details of CMSL constructs.

The language consists of a number of major syntactic constructs, each of which is enclosed with its own unique keyword and common end marker, for example:

```
CIM_SPEC
...
END
```

Within the bounds of a keyword/end marker, ‘attributes’ can be declared to specify the details of the major construct. Parameters can also be associated with these attributes to define maximum/minimum values, default values, optional values, number of duplicate attribute occurrences, etc.

Six major CMSL syntactic constructs have been identified. Some of these are described in the example illustrated in Fig 5. It should be noted that CMSL reserved words are shown in **BOLD** characters.

Figure 5 illustrates how a typical CMSL structure and the representative CMSL could be designed. Parts (a), (b) and (c) illustrate how the final structure is built up. The structure in this example, while not necessarily complete, could be used to define movie type offerings. Each structure equates to a specific set of CMSL constructs. For instance, structure (a) defines the actual CIM_SPEC ‘movie’ including the associated generic data component. The order of CMSL constructs in Fig 5 is that required by the CMSL interpreter.

Structure (b) illustrates the inclusion of a number of attribute components — namely ‘actors’, ‘rating’, ‘genre’ and ‘price’ attributes. The dashed line between the movie CI and genre attribute components indicates that the relationship is optional; this is reflected in the CMSL by the

keyword ‘**OPTIONAL**’ in the **ATTRIBUTES_USED** construct. The use of an ‘optional’ keyword aids the automated testing process to determine if an item is complete, i.e. it can check for completeness of metadata for a given piece of content by comparing entries in the library against the CMSL specification to which the item was built. The ‘ratings’ attribute also illustrates the capability of CMSL to specify default values and a list of valid values, both of which improve efficiency and accuracy during metadata acquisition. In addition, they provide another level of confidence with respect to the integrity of the content’s metadata. The relationship between the movie CI and the ‘Actors’ attribute component is a one-to-many relationship, as indicated by the branched link. This indicates that a piece of content built to this specification may have many Actors assigned to it. For example, the *Star Wars* offering may have ‘Harrison Ford’, ‘Carrie Fisher’ and ‘Alec Guinness’ assigned to it. Again this is reflected in the CMSL by the keyword ‘**OCCURS**’. The numbers following ‘**OCCURS**’ further defines the relationship, i.e. in the example given it indicates that there must be at least one value (an actor) assigned, and a maximum of five values.

Structure (c) indicates the inclusion of a number of associations. The MPEG ‘Trailer’ association is again indicated as optional.

3.3 Generation of CMSL

The example structure illustrated in Fig 5 is a simple one. Far more complex structures are possible. The flexibility in defining these structures is vast and is dependent upon the designer’s expertise. As the structures become more complex, there arises a real need to manage them in a user friendly environment. In addition, the production of the appropriate CMSL could be both a potential bottle-neck to the system and error prone.

By providing an object-oriented user-interface tool that allows the user to build their required CMSL structures in a drag-and-drop environment, it may be possible to overcome these problems. The tool could be used to automatically generate the CMSL. This approach offers a number of key advantages:

- provision of a user-friendly environment for designing and updating CMSL structures,
- management of CMSL structure definitions promoting reuse across specifications,
- reduction in the required level of expertise, by abstracting them from the internal complexities of CMSL as a language,

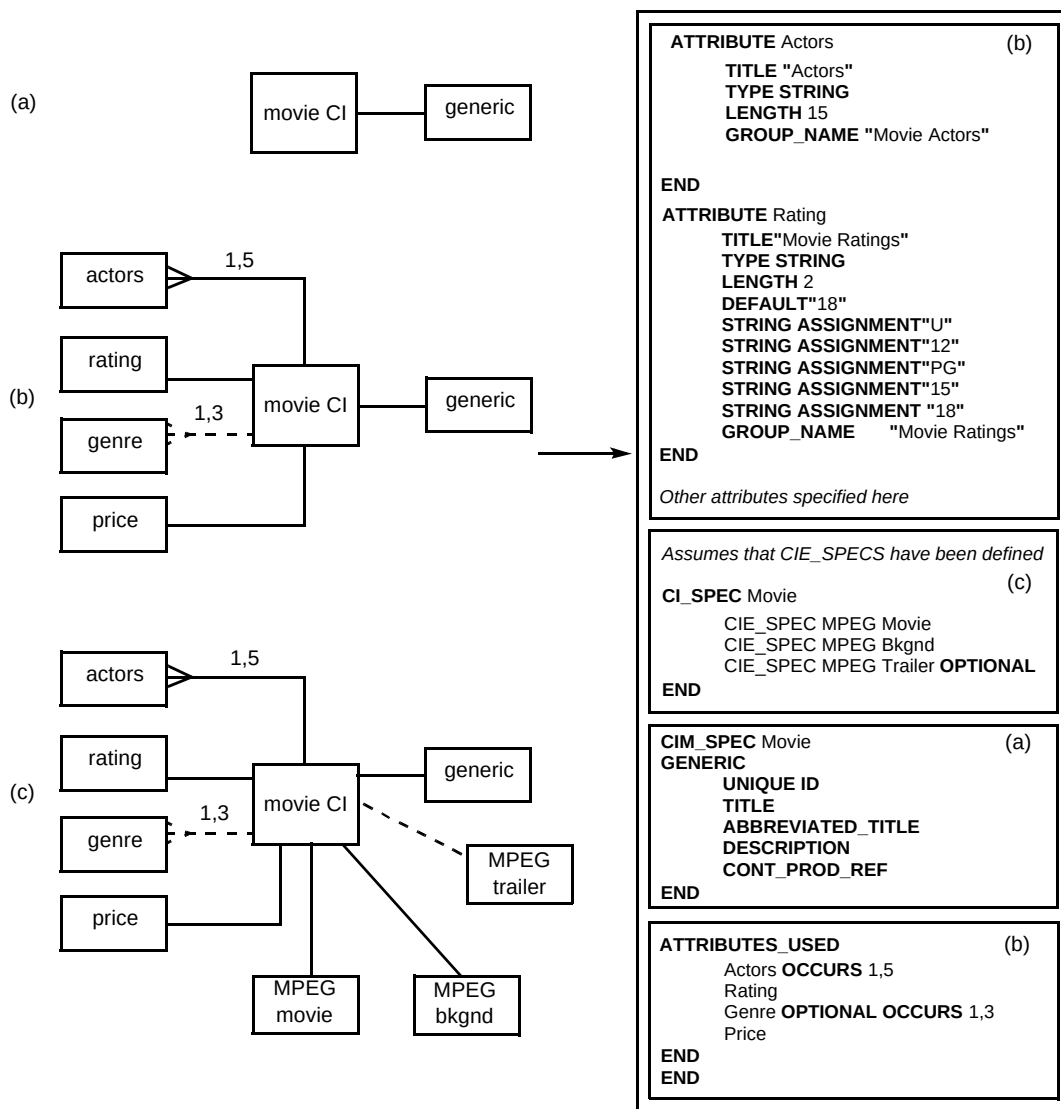


Fig 5 CMSL structures and the representative CMSL.

- reduction in the possibility of invalid structure definitions which would result in invalid CMSL — these types of error are particularly dangerous, as they typically propagate through the system,
- automation of the generation of CMSL, hence reducing human error and speeding up the process.
- it must provide the ability to include standards-based and proprietary, read-only, component libraries (e.g. DAVIC-defined components),
- it should provide persistence for components in a networked, multi-user environment — this will require some form of session management to ensure that only one person can edit a common component at any one time, other users, however, being able to view a read-only version of the component (security issues could also be addressed via the session manager),

3.4 Requirements of the tool

The preliminary requirements for such a tool are identified below :

- it must allow the creation of component libraries, allowing reuse of components (i.e. attributes, CI specifications, etc) across several content specifications,
- it should provide a feature-rich environment, i.e. drag-and-drop, automatic generation of CMSL, capability to view generated CMSL, etc,

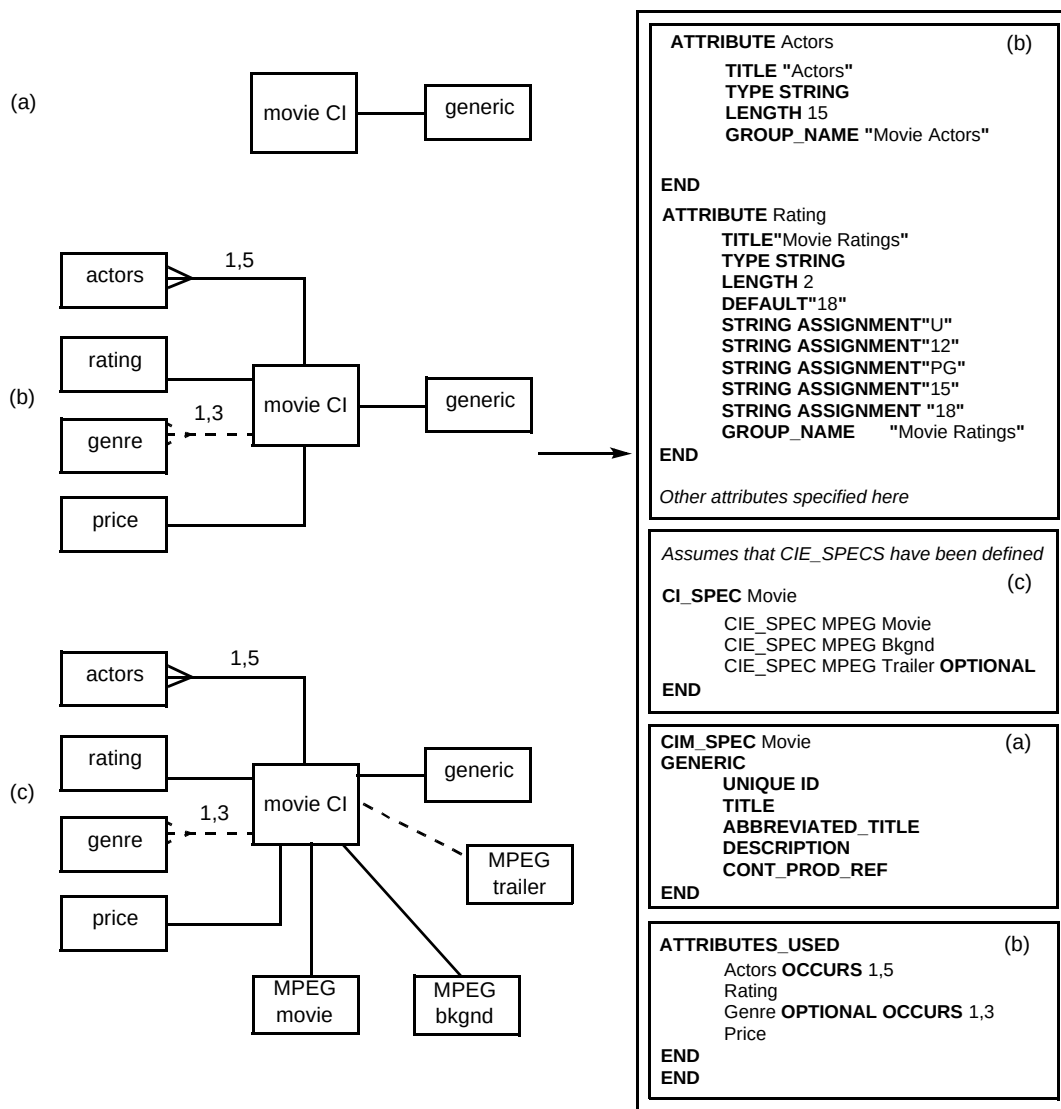


Fig 5 CMSL structures and the representative CMSL.

- reduction in the possibility of invalid structure definitions which would result in invalid CMSL — these types of error are particularly dangerous, as they typically propagate through the system,
- automation of the generation of CMSL, hence reducing human error and speeding up the process.
- it must provide the ability to include standards-based and proprietary, read-only, component libraries (e.g. DAVIC-defined components),
- it should provide persistence for components in a networked, multi-user environment — this will require some form of session management to ensure that only one person can edit a common component at any one time, other users, however, being able to view a read-only version of the component (security issues could also be addressed via the session manager),

3.4 Requirements of the tool

The preliminary requirements for such a tool are identified below :

- it must allow the creation of component libraries, allowing reuse of components (i.e. attributes, CI specifications, etc) across several content specifications,
- it should provide a feature-rich environment, i.e. drag-and-drop, automatic generation of CMSL, capability to view generated CMSL, etc,

- it should provide the capability of grouping attribute and association components into a single 'group' object so that content specifications can share common groups which can be assigned as a single object, hence reducing the user's workload, simplifying design and reducing the possibility of errors,
- it should provide on-line validation — clearly an order exists in combining components to create new content specification, e.g. it would not be possible to assign an 'Actors' attribute component to a 'Movie' offering content specification if the 'Actors' attribute component did not exist; this is intuitive in a drag-and-drop environment,
- it should provide a hierarchical windowing environment — users should be able to edit individual components by selecting them from any window in which they exist (or a link to them exists), e.g. when editing a content specification, the user may select an assigned attribute, which should invoke another edit window displaying the selected attribute object; the user may then edit the attribute at will.

4. Conclusions

Considerable progress has been made in tackling the challenges relating to content production and delivery processes. A generic tool-set has been defined which is capable of operating in a distributed, heterogeneous environment. The concept demonstrator has been implemented using the Java programming language [4], which provides cross-platform portability. This will be used to further refine these process interfaces.

The aim now is to downstream the achievements of the project into new multimedia services to provide the benefits of reduced development costs and time-scales, thus allowing the provision of quality services which end users have come to expect. The increased automation of processes has contributed significantly to the benefits which have been gained, and should assist in reducing service operating costs.

The next logical step is the extension and refinement of CMSL to encompass definitions of higher level content items and hence determine the scope or extent to which CMSL can be used. This will be determined by the interpretation of the term 'content'.

An obvious piece of content is a video clip; however, as the interpretation becomes more abstract, so do the potential

uses of CMSL. For example, a Web page within a Web site could be regarded as a piece of content, which itself contains content items. Furthermore, the actual service may be regarded as a piece of content which is displayed via a Web browser.

This leads to the philosophy that: 'Content is everything, and everything is content.'

It is hoped that CMSL will be adopted as an open, standardised content-specification language by the vendors of proprietary application-authoring and content-authoring tools to enable the efficient management of content. Without such automated processes, high-volume multimedia services cannot be cost effective.

References

- 1 Kerr G W: 'A review of BT's interactive TV trials', IEE Colloquium on Interactive Television, London, pp 6/1-6/5 (October 1995).
- 2 Withnell J: 'BT Interactive TV delivering multimedia services to the home', IEE Colloquium on The Impact of Multimedia Services on the Home Environment, London, pp 2/1-2/2 (January 1996).
- 3 Searby S et al: 'The authoring and automation of content with its associated metadata for multimedia on-line services', British Telecommunications Eng J (to be published).
- 4 Java, <http://java.sun.com/allabout.html>
- 5 'Digital Audio Video Council 1.1 Specification Part 1 - Description of DAVIC Functionality' (March 1996), <http://davic.www.org>
- 6 'Information Resources for CORBA and the OMG', <http://www.acl.lanl.gov/CORBA/>



Royston Walker joined BT Laboratories in 1994 having obtained a BEng in electronic engineering from the University of Hull and an MSc in Telecommunications and Information Systems from the University of Essex.

Since then he has been involved with the development of multimedia services working on BT Interactive TV and BT Touchpoint.

He is currently a member of the on-line services unit investigating management and delivery mechanisms for multimedia content as a part of the IMS futures campaign.

- it should provide the capability of grouping attribute and association components into a single 'group' object so that content specifications can share common groups which can be assigned as a single object, hence reducing the user's workload, simplifying design and reducing the possibility of errors,
- it should provide on-line validation — clearly an order exists in combining components to create new content specification, e.g. it would not be possible to assign an 'Actors' attribute component to a 'Movie' offering content specification if the 'Actors' attribute component did not exist; this is intuitive in a drag-and-drop environment,
- it should provide a hierarchical windowing environment — users should be able to edit individual components by selecting them from any window in which they exist (or a link to them exists), e.g. when editing a content specification, the user may select an assigned attribute, which should invoke another edit window displaying the selected attribute object; the user may then edit the attribute at will.

4. Conclusions

Considerable progress has been made in tackling the challenges relating to content production and delivery processes. A generic tool-set has been defined which is capable of operating in a distributed, heterogeneous environment. The concept demonstrator has been implemented using the Java programming language [4], which provides cross-platform portability. This will be used to further refine these process interfaces.

The aim now is to downstream the achievements of the project into new multimedia services to provide the benefits of reduced development costs and time-scales, thus allowing the provision of quality services which end users have come to expect. The increased automation of processes has contributed significantly to the benefits which have been gained, and should assist in reducing service operating costs.

The next logical step is the extension and refinement of CMSL to encompass definitions of higher level content items and hence determine the scope or extent to which CMSL can be used. This will be determined by the interpretation of the term 'content'.

An obvious piece of content is a video clip; however, as the interpretation becomes more abstract, so do the potential

uses of CMSL. For example, a Web page within a Web site could be regarded as a piece of content, which itself contains content items. Furthermore, the actual service may be regarded as a piece of content which is displayed via a Web browser.

This leads to the philosophy that: 'Content is everything, and everything is content.'

It is hoped that CMSL will be adopted as an open, standardised content-specification language by the vendors of proprietary application-authoring and content-authoring tools to enable the efficient management of content. Without such automated processes, high-volume multimedia services cannot be cost effective.

References

- 1 Kerr G W: 'A review of BT's interactive TV trials', IEE Colloquium on Interactive Television, London, pp 6/1-6/5 (October 1995).
- 2 Withnell J: 'BT Interactive TV delivering multimedia services to the home', IEE Colloquium on The Impact of Multimedia Services on the Home Environment, London, pp 2/1-2/2 (January 1996).
- 3 Searby S et al: 'The authoring and automation of content with its associated metadata for multimedia on-line services', British Telecommunications Eng J (to be published).
- 4 Java, <http://java.sun.com/allabout.html>
- 5 'Digital Audio Video Council 1.1 Specification Part 1 - Description of DAVIC Functionality' (March 1996), <http://davic.www.org>
- 6 'Information Resources for CORBA and the OMG', <http://www.acl.lanl.gov/CORBA/>



Royston Walker joined BT Laboratories in 1994 having obtained a BEng in electronic engineering from the University of Hull and an MSc in Telecommunications and Information Systems from the University of Essex.

Since then he has been involved with the development of multimedia services working on BT Interactive TV and BT Touchpoint.

He is currently a member of the on-line services unit investigating management and delivery mechanisms for multimedia content as a part of the IMS futures campaign.



Paul Foster started his career in BT as an apprentice in 1973 and then, with a BT award, gained a 1st class honours degree in Computer Science at Leicester Polytechnic.

Since 1983 he has worked at BT Laboratories and has developed systems and software for photovideotex servers, Talking Pages and BT's Directory Enquiries teleworking experiment.

For the past three years he has worked on multimedia systems, in particular air line information services (ALIS) and IMS futures.



Steve Banthorpe joined BT Laboratories (BTL) in 1983 after 12 years experience in the TV/video retail industry. He has HNC qualifications in both electrical engineering and telecommunications. He initially worked in the visual applications unit on the design of the first generation videoconferencing products. He moved to Visual and Broadcast Services in 1988 as technical support and training manager to VC operations units.

In 1991 he became a part of the product management team in London as product manager for the VC5000 range of systems.

He was responsible for the design and introduction of the on-screen GUI on BT's System 3 range. In 1995 he returned to BTL to capture requirements for the back-end interfaces for the BTiTV trial in Ipswich and Colchester. This progressed to leading the team which produced a range of content management tools for internal and external BTiTV services. He is currently working in the On-line Services unit on the IMS futures project.



Paul Foster started his career in BT as an apprentice in 1973 and then, with a BT award, gained a 1st class honours degree in Computer Science at Leicester Polytechnic.

Since 1983 he has worked at BT Laboratories and has developed systems and software for photovideotex servers, Talking Pages and BT's Directory Enquiries teleworking experiment.

For the past three years he has worked on multimedia systems, in particular air line information services (ALIS) and IMS futures.



Steve Banthorpe joined BT Laboratories (BTL) in 1983 after 12 years experience in the TV/video retail industry. He has HNC qualifications in both electrical engineering and telecommunications. He initially worked in the visual applications unit on the design of the first generation videoconferencing products. He moved to Visual and Broadcast Services in 1988 as technical support and training manager to VC operations units.

In 1991 he became a part of the product management team in London as product manager for the VC5000 range of systems.

He was responsible for the design and introduction of the on-screen GUI on BT's System 3 range. In 1995 he returned to BTL to capture requirements for the back-end interfaces for the BTiTV trial in Ipswich and Colchester. This progressed to leading the team which produced a range of content management tools for internal and external BTiTV services. He is currently working in the On-line Services unit on the IMS futures project.